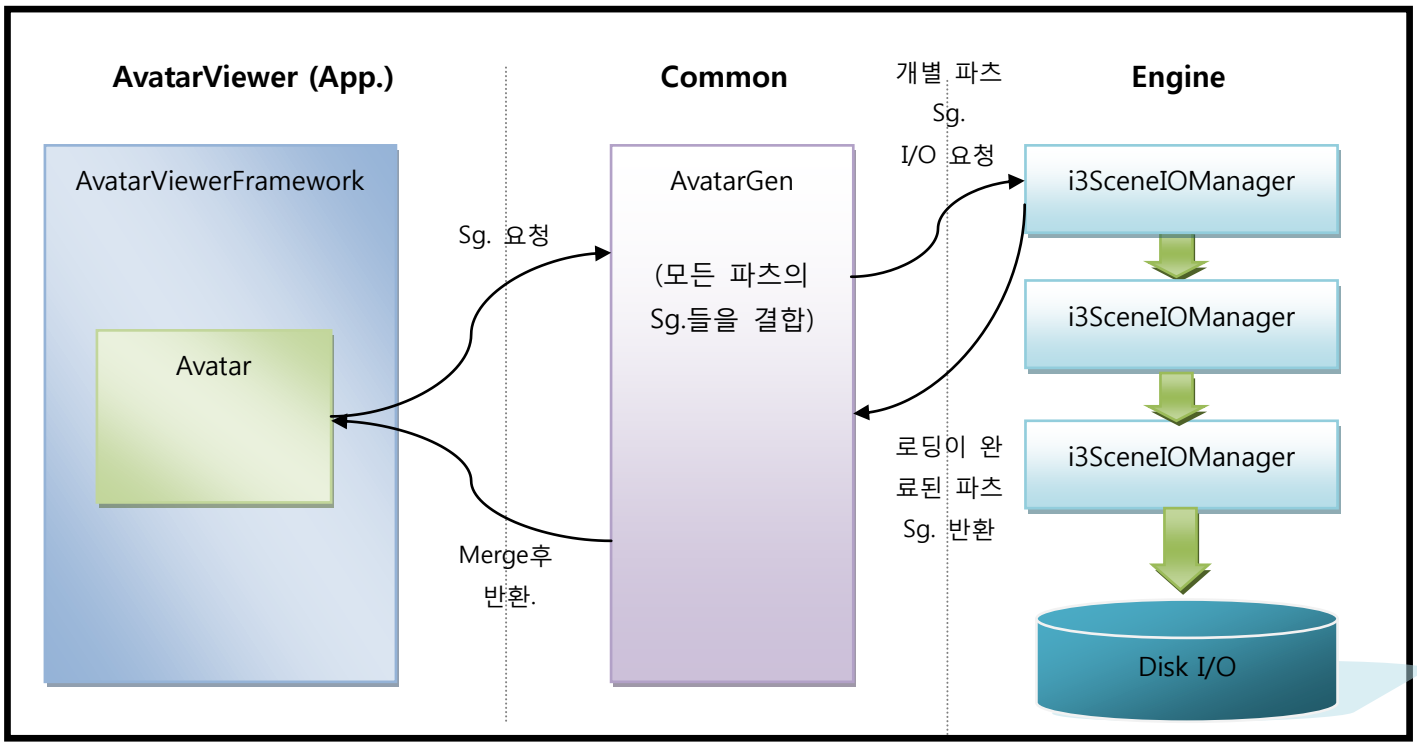
Avatar System : AvatarViewer

작성자 : 이진균 (jinkyun.lee@zepetto.com)

주요 Class



AvatarGen

Category	Base class	Path
Common	**i3ElementBase**	<PointBlank>/Source_New/Avatar

실질적인 Background Loading을 통해, Avatar용 Scene-Graph를 생성해 주는 Class. 각 파츠에서 사용되는 Texture와 Geometry를 모두 합치는 처리를 수행한다.

Avatar

Category	Base class	Path
App.	**i3Chara**	<PointBlank>/Source_New/AvatarViewer

AvatarViewer 프로그램에서 화면에 보이는 캐릭터를 위해 생성되는 객체. 실질적인 Avatar 관련 기능을 직접 제공하는 것은 없으며, AvatarGen class에 의해 생성된 Sg.를 보여주는 역할만 한다.

AvatarViewerFramework

Category	Base class	Path
App.	**i3ViewerFramework**	<PointBlank>/Source_New/AvatarViewer

AvatarViewer 프로그램을 위한 Framework class.
i3Viewer2 프로그램의 Framework와 별반 차이가 없다.

Avatar Class를 하나 생성하여 관리한다.

i3SceneIOManager

Category	Base class	Path
Engine	**i3ElementBase**	<Engine>/src/i3Scene <Project>/i3SceneDx/Background Loading

외부에서 Background Sg. Loading을 위해 사용하는 Class로, 관련 Queue와 Thread를 관리한다.
Engine에서 자동으로 생성하지 않기때문에, Background Loading을 위해서는 App.에서 반드시 생성하여야 한다.
AvatarViewer 프로그램에서는 AvatarViewerFramework가 생성하고 있다.

i3SceneIOQueue

Category	Base class	Path
Engine	**i3ElementBase**	<Engine>/src/i3Scene <Project>/i3SceneDx/Background Loading

i3SceneIOManager와 i3SceneIOThread 간의 I/O Request Queue 역할을 하는 Class. 256개의 Queue를 가지고 있으며, Ring-Buffer 방식으로 동작한다. i3SceneIOManager와 i3SceneIOThread 간의 Synchronization 기능 역시 제공한다.

i3SceneIOThread

Category	Base class	Path

Engine	**i3Thread**	<Engine>/src/i3Scene <Project>/i3SceneDx/Background Loading

실질적인 Background Scene-Graph Loading을 처리하는 Class.

Resources

Avatar System에서 각 파츠를 올바르게 읽어오기 위해서는 일정한 규칙하에 관리되어야 한다.

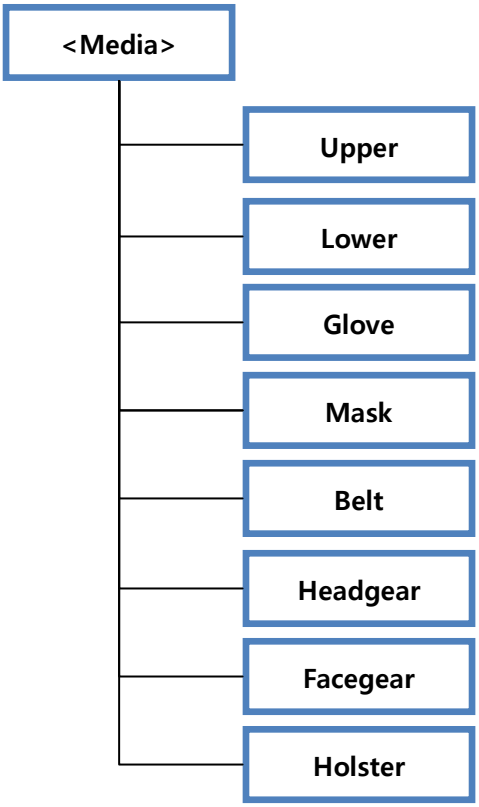
Parts

PB용 Parts는 아래와 같이 구성되며, Resource의 특징 등을 정리하면 아래와 같다.

Parts Name	Skinning	1인칭	특성	Attach Node
Upper	O	O	다른 모든 파츠의 대표격이기 때문에 완전한 Bone 구조를 가지고 있어야 한다.	-
Lower	O	X		-
Mask	O	X		-
Glove	O	O		-
Belt	X	X		Bone_BeltAttach
Headgear	X	X		Bone_HeadGearAttach
Facegear	X	X		Bone_FaceGearAttach
Holster	X	X		Bone_HolsterAttach

Folder 구조

모든 파츠들은 아래의 Folder 구조에 배치되어야 한다.



Naming Rule

각 폴더에 배치된 파츠 파일들은 아래의 규칙으로 이름이 명명되어야 한다.

```
<Name>_LOD#.i3s
```

ex.) DFox_Male_LOD0.i3s, DFox_Male_LOD1.i3s

LOD0은 1인칭 시점의 파츠이며, LOD1~3은 3인칭 시점의 파츠다.

Usages

파츠 변경

AvatarViewer 프로그램에서 i3s 파일을 Drop하면 해당 파일의 경로에 포함된 폴더명으로 어느 파츠인지 구분하여 재구성한다. 이에 해당하는 코드는 AvatarViewerFramework::LoadPart() 함수에 있다.

아래는 파츠를 변경하는 예제 코드이다.

```
Avatar * pAvatar;

pAvatar->setPartName( AVATAR::PART_UPPER, "Redbulls_Male");
```

setPartName()이 호출되면 Avatar Class는 단순히 자신이 가지고 있는 AvatarGen class의 동일 함수를 호출한다. 예제와 같이 "Redbulls_Male"이란 이름으로 AvatarGen::setPartName() 함수가 호출되면 내부적으로 아래의 파일들을 읽어 들인다.

1. <Media>/Avatar/Upper/Redbulls_Male_LOD0.i3s
2. <Media>/Avatar/Upper/Redbulls_Male_LOD1.i3s
3. <Media>/Avatar/Upper/Redbulls_Male_LOD2.i3s
4. <Media>/Avatar/Upper/Redbulls_Male_LOD3.i3s

이 중, 존재하는 i3s만 읽어와 구성한다.

Loading이 끝난 시점 확인

아래의 코드는 Avatar Class의 OnUpdate해서 처리하는 부분이다.

```
void Avatar::OnUpdate( REAL32 tm)
{
        i3Chara::OnUpdate( tm);

        if( m_pGen != NULL)
        {
                // Avatar Gen의Update 호출.
                // 여기에서는항상호출하고있지만, 사실Loading이일어나고있지않다면호출될필요없다.
                m_pGen->OnUpdate( tm);

                // 모든Loading이끝나고, Geometry 및Texture에대한재구성이모두완료되면
                // isReadyScene() 함수가true를반환한다.
                if( m_pGen->isReadyScene())
                {
                        // AvatarGen::GetGeneratedBody() 함수가한번호출되고나면
                        // AvatarGen은자신이생성했던Sg.를가지고있지않는다.
                        m_pBody = m_pGen->GetGeneratedBody();

                        // 이하의코드는App.에국한되는코드.
                        // 반드시처리해야하는코드가아님.
                        m_pBody->AddRef();

                        Create( m_pBody);

                        m_pBody->Release();

                        ...
                }
        }

}
```

Background Loading

i3Engine 내부에서 i3s 파일을 Background Loading 할 때, Foreground Loading과는 다른 점이 있다.

이러한 특징이 발생한 가장 큰 이유는 DirectX 9이 Multi-Threading을 원숙하게 지원하지 않기 때문이다. 실제로 DirectX Object를 생성하지 않은 Thread에서 DirectX 관련 객체들, 예를 들어 Texture 등을 생성하거나 할 경우, Warning이 발생하며, 오동작을 일으키는 원인이 되기도 한다.

i3Engine에서 i3Texture와 i3VertexArray, i3IndexArray 등의 Gfx Resource들은 DirectX 객체를 직접 사용하는 Class 들이기 때문에, Background로 i3s 파일을 로딩하는 중에 문제가 발생할 수 있다.
그것을 피하기 위해서 Background Loading으로 i3s 파일을 읽어 들일 때에는, 포함된 i3Texture 등을 아래와 같은 Ghost Class로 대체 생성한다.

Abstract Class	Concrete Class	Ghost Class
i3Texture	i3TextureDX	i3TextureMem
i3VertexArray	i3VertexArrayDX	i3VertexArrayMem
i3IndexArray	i3IndexArrayDX	i3IndexArrayMem

Ghost Class들은 Concrete Class와 동일한 Interface 및 관리 기능을 제공하지만, DirectX 객체를 전혀 사용하지 않기 때문에 Background Thread에서 자유롭게 생성할 수 있기 때문이다.
문제는 이들 Ghost Class들은 DirectX에서 직접 사용될 수 없으며, 반드시 Concrete Class로 대체되어야만 실질적인 Rendering에 활용될 수 있다.

Avatar System에서는 Ghost Class로 생성된 Scene-Graph를 아래에서 설명할 Geometry 및 Texture 재구성 작업 단계에서 Concrete Class로 대체 생성한다.

Loading이 완료된 시점에서 각 파츠들을 재구성하는 것은 AvatarGen::BuildScene() 함수에서 이루어진다.

Geometry 재구성

AvatarGen은 Loading된 모든 파츠들을 재구성하여 i3Body Node를 Root로 하는 새로운 Scene-Graph를 생성해준다.

AvatarGen::GetGeneratedBody() 함수를 통해 생성된 Sg.를 얻어올 수 있다.

Geometry에 대한 재구성은 AvatarGen::BuildShape() 함수에서 이루어진다. Rendering 성능을 향상시키기 위해, Skinning의 여부와 무관하게 모든 파츠의 Geometry를 Skinning을 사용하는 하나의 Geometry로 구성한다. 이것은 Draw call count를 줄이기 위한 것인데, 이를 위해 Geometry뿐만 아니라 Texture 역시 하나의 Texture로 통합 생성된다. Texture의 재구성에 대한 자세한 설명은 Texture 재구성 섹션을 참고하기 바란다.

NOTE: **Skinning을 위한 기본 Bone Hierarchy는 Upper 파츠의 LOD1에서 가져온다**.
다시 말해, Upper LOD1 파츠는 반드시 모든 Bone 정보를 포함할 필요가 있다.

Texture 재구성

Avatar System에서 각 파츠들은 조합되어 재구성된다. 이 중, Texture도 예외가 아닌데, 하나의 2048 x 2048 크기의 Texture로 합쳐진다. 물론 이것은 Diffuse Map 하나에 대한 것이며, Normap Map 등과 같은 용도 구분에 따라 3장이 추가로 만들어진다.
각 용도별 텍스쳐의 크기 비율은 아래와 같다.

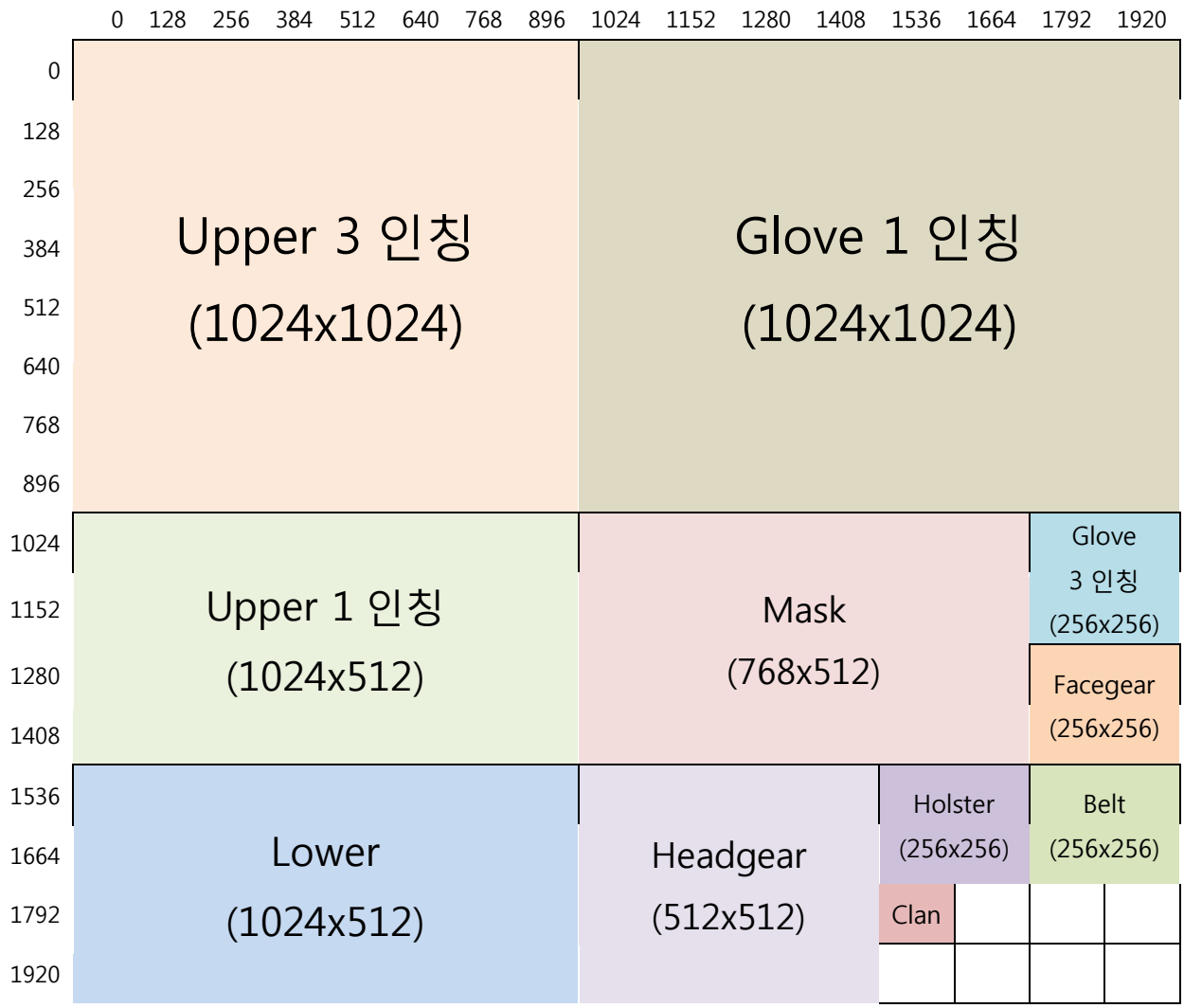
Textures	Width	Height	Rate
Diffuse Map	2048	2048	100%
Normal Map	2048	2048	100%
Specular Map	1024	1024	50%
Reflect Mask Map	1024	1024	50%

이들 텍스쳐의 크기는 그래픽 옵션에 "상"에 해당하는 경우의 것이며, "중"과 "하"로 내려갈 때마다 50%씩 떨어진다.

하나로 합쳐지는 이유로, 각 파츠별 사용 가능 텍스쳐의 크기 역시 규칙이 적용되고 있으며, 아래와 같다.

Parts	1인칭 Size	3인칭 Size
Upper	1024 x 512	1024 x 1024
Lower	-	1024 x 512
Mask	-	768 x 512
Glove	1024 x 1024	256 x 256
Headgear	-	512 x 512
Facegear	-	256 x 256
Belt	-	256 x 256
Holster	-	256 x 256
Clan mark	-	128 x 128

각 파츠별 텍스쳐가 통합될 때의 위치는 아래와 같다.



위와 같은 배치는 AvatarGen::BuildMacroTex() 함수에서 이루어진다.